

Advanced Database Management Systems (CSC-234)

Introduction to NoSQL

SQL Systems

ACID: Atomicity, Consistency, Isolation, and Durability

NoSQL

BASE: Basically Available, Soft State, Eventually Consistent

ACID: Atomicity

- Guarantee that either all of the transaction succeeds or none of it does
- E.g., transferring from checking account into saving account
 - Complete all steps as a single indivisible unit or complete none of them

ACID: Consistency

- Strict consistency required in relational databases
- A transaction does not leave the database in a state that violates the integrity of the data

ACID: Consistency

- Transferring \$100 from your savings account to your checking account must end with either
 - (a) \$100 more in your checking account and \$100 less in your savings account or
 - (b) both accounts have the same amount as they had at the start of the transaction
- Consistency ensures no other possible state could result after a transfer operation

ACID: Isolation

- Guarantees that all transactions will occur in isolation
- No transaction will be affected by any other transaction

ACID: Durability

- This means that once a transaction or operation is completed, it will remain even in the event of a power loss
- In effect, this means that data is stored on disk, flash, or other persistent media

RELATIONAL DATABASE MANAGEMENT SYSTEMS ARE DESIGNED TO SUPPORT ACID TRANSACTIONS. NOSQL DATABASES TYPICALLY SUPPORT BASE TRANSACTIONS, ALTHOUGH SOME NOSQL DATABASES ALSO PROVIDE SOME LEVEL OF SUPPORT FOR ACID TRANSACTIONS.

BASE: Basically Available

- There can be a partial failure in some parts of the distributed system and the rest of the system continues to function
- For example, if a NoSQL database is running on 10 servers without replicating data and one of the servers fails, then 10% of the users' queries would fail, but 90% would succeed
- NoSQL databases often keep multiple copies of data on different servers. This allows the database to respond to queries even if one of the servers has failed.

BASE: Soft State

- It is okay for data to be eventually be over-written by more recent data

BASE: Eventually Consistent

- There may be times when the database is in an inconsistent state
- E.g., multiple copies exist but there can be situations in which they are not consistent for short periods of time

Four Types of NoSQL Systems

- Key-value pair databases
- Document databases
- Column family store databases
- Graph databases

On distributed systems

- It's not necessary for noSQL systems to be distributed
- However, some of the most appealing features do require a distributed deployment
- When availability and scalability are top concerns distributed deployments make sense

With distributed systems

- Decisions and trade-offs required not found in single-server implementations
- Balance need for scalability, availability, consistency, partition protection, and durability

Key-Value Pair Databases

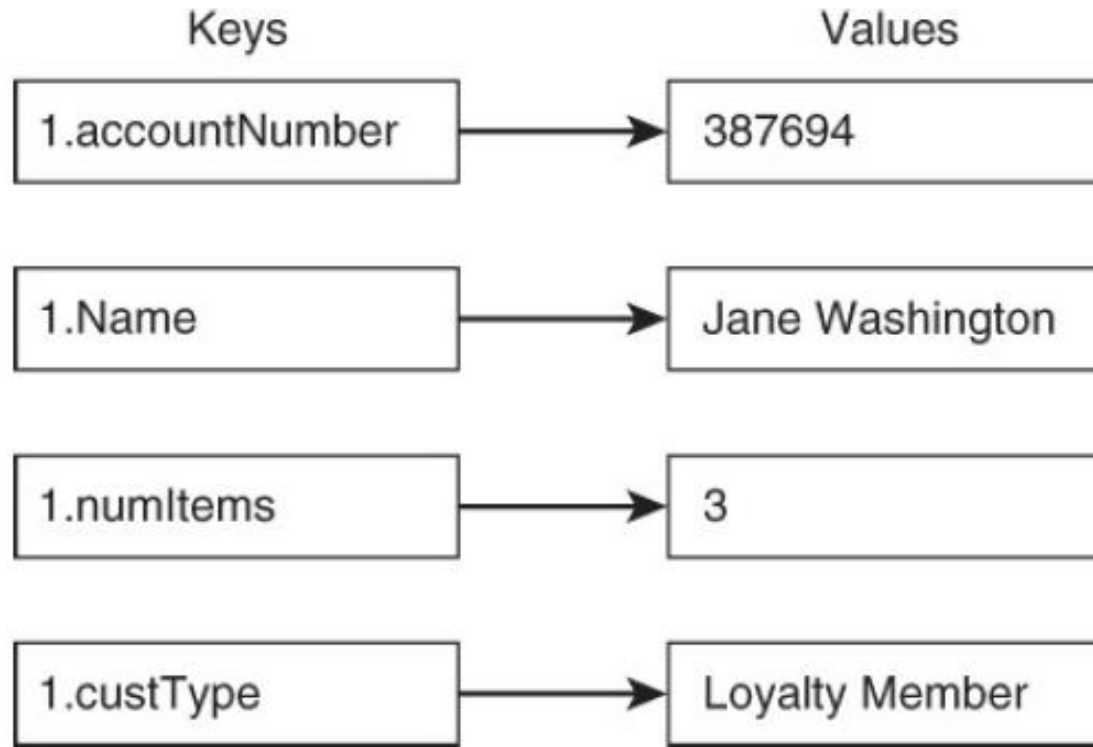


Figure 2.11 *Key-value databases are modeled on a simple, two-part data structure consisting of an identifier and a data value.*

All of these values are associated with a customer, so you can generate a sequential number for each customer. For each item you are storing, you create a new key by appending the name of the item you are storing to the customer number.

Key-Value Pair Databases

- Approach works for relatively simple databases

Key-Value Databases

- In a database with customer and warehouse information for example, use pre-fixes to avoid clashes

- cust1.accountNumber
- cust1.name
- cust1.address
- cust1.numItems
- cust1.custType
- cust2.accountNumber
- cust2.name
- cust2.address
- cust2.numItems
- cust2.custType

- wrhs1.number
- wrhs1.address
- wrhs2.number
- wrhs2.address

Different Namespaces using Buckets

Database

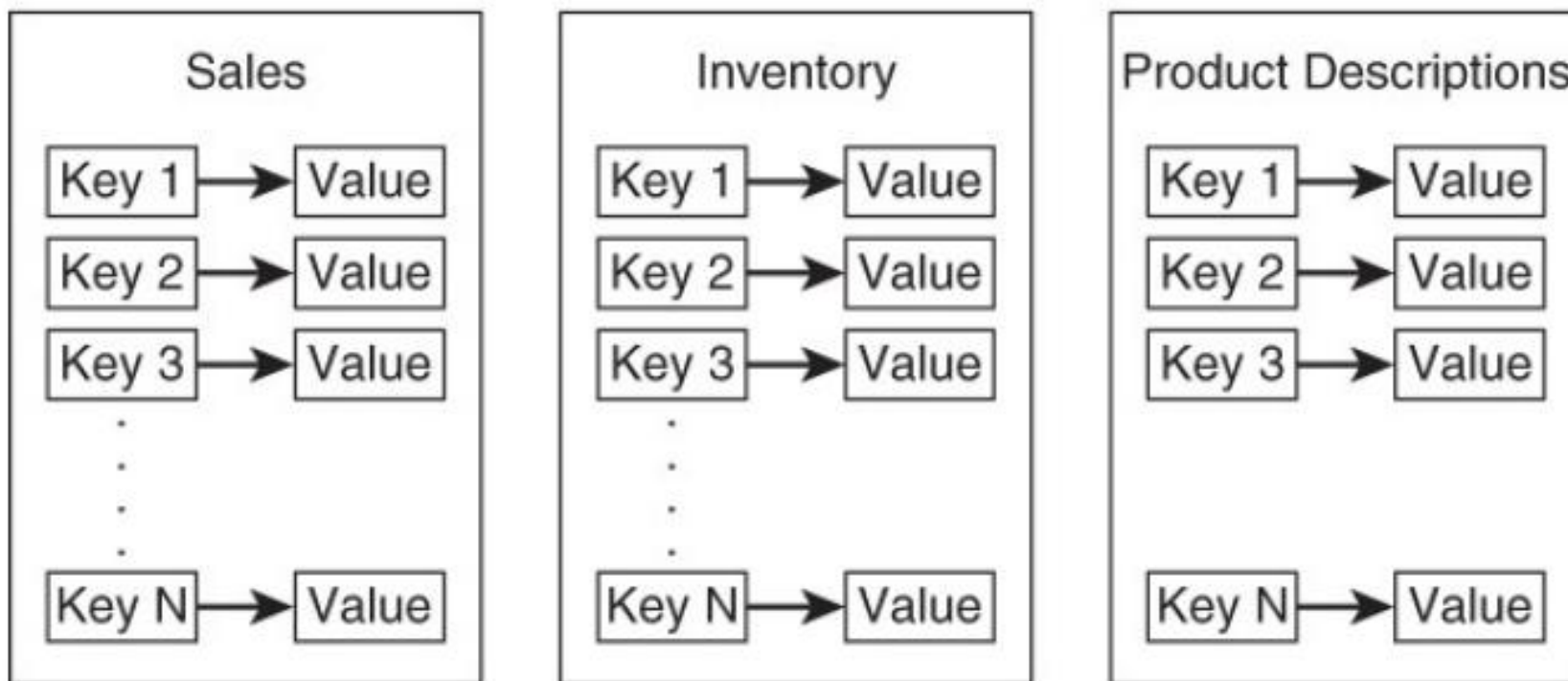


Figure 2.12 Key-value databases may support separate namespaces within a single database.

Values

- Values can range from strings and numbers to images and binary objects

Key Values vs. Relational

- Key-values based on minimal principles for storing and retrieving data
- No tables, no columns, no keys, no joins ...
- No powerful query mechanism like SQL

Key Values vs. Relational

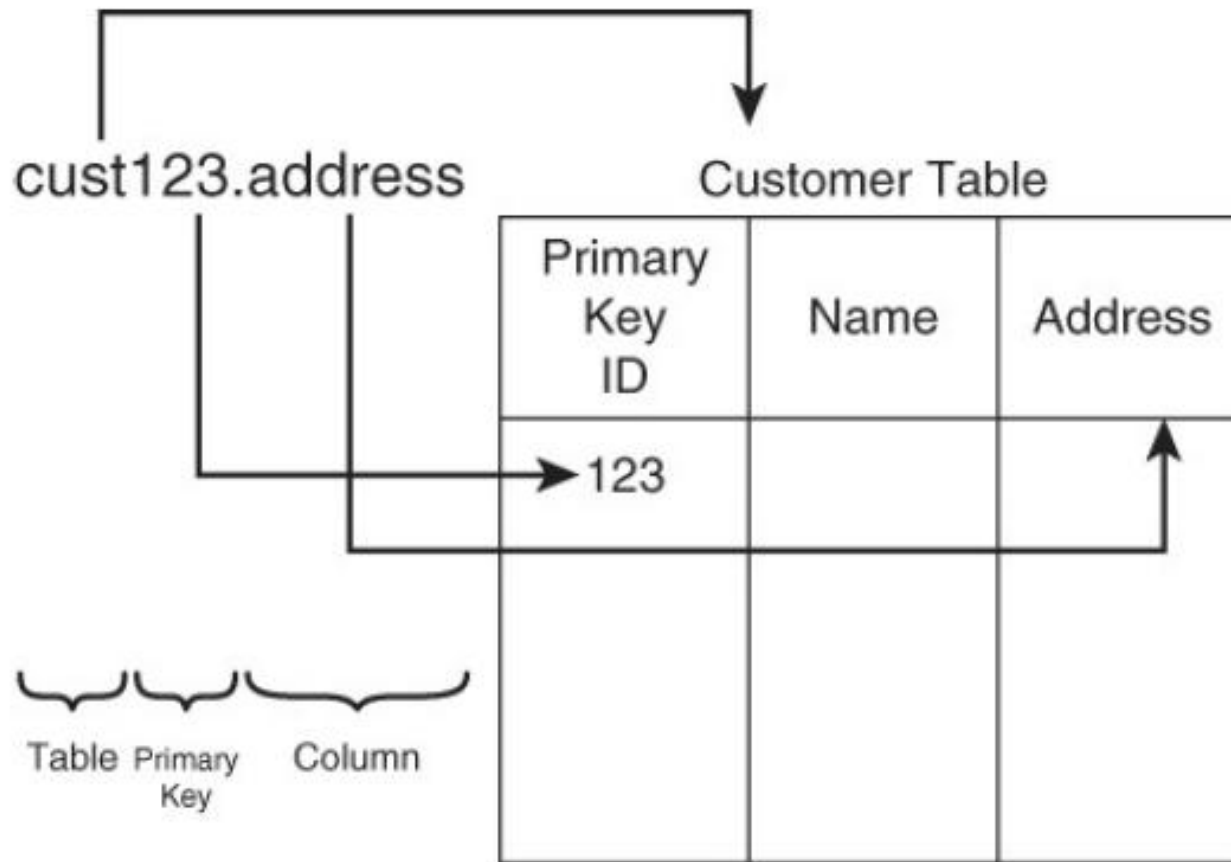


Figure 2.13 *The key-naming convention outlined above maps to patterns seen in relational database tables.*



Join us at [Redis Day London 2019](#) full day conference featuring user stories and developer updates from the Redis world.

Redis is an open source (BSD licensed), in-memory data structure store, used as a database, cache and message broker. It supports data structures such as strings, hashes, lists, sets, sorted sets with range queries, bitmaps, hyperloglogs, geospatial indexes with radius queries and streams. Redis has built-in replication, Lua scripting, LRU eviction, transactions and different levels of on-disk persistence, and provides high availability via Redis Sentinel and automatic partitioning with Redis Cluster. [Learn more](#) →

Try it

Ready for a test drive? Check this [interactive tutorial](#) that will walk you through the most important features of Redis.

Download it

[Redis 5.0.5 is the latest stable version.](#) Interested in release candidates or unstable versions? [Check the downloads page.](#)

Quick links

Follow day-to-day Redis on [Twitter](#) and [GitHub](#). Get help or help others by subscribing to [our mailing list](#), we are 5,000 and counting!

Document Databases

- Uses a key value approach but stores documents as values
- Documents are semi-structured entities in standard formats such as
 - JSON (JavaScript Extended Object Notation)
 - XML (Extensible Markup Language)

Documents

- Instead of storing each attribute with a separate key, document databases hold multiple attributes in a single document

Document in JSON Format

```
{  
  firstName: "Alice",  
  lastName: "Johnson",  
  position: "CFO",  
  officeNumber: "2-120",  
  officePhone: "555-222-3456",  
}
```

Document Databases

- Fixed schema is not required before you can begin adding data to the database
- Simply adding a document to the database creates the underlying data structures needed to support the document
- Therefore, there is more flexibility

```
{  
  firstName: "Alice",  
  lastName: "Johnson",  
  position: "CFO",  
  officeNumber: "2-120",  
  officePhone: "555-222-3456",  
}
```

```
{  
  firstName: "Bob",  
  lastName: "Wilson",  
  position: "Manager",  
  officeNumber: "2-130",  
  officePhone: "555-222-3478",  
  hireDate: "1-Feb-2010",  
  terminationDate: "12-Aug-2014"  
}
```

Two valid entries in the same document database.

**SOFTWARE APPLICATIONS
WILL HOWEVER BE
RESPONSIBLE FOR
HANDLING THESE
DISCREPANCIES**

Querying

```
db.employees.find( { position:"Manager" })
```

XML Format

```
<?xml version="1.0"?>
- <job>
  - <production>
    <ApprovalType>WebCenter</ApprovalType>
    <Substrate>carton 150 gr</Substrate>
    <SheetSize>220-140</SheetSize>
    <press>SuperFlat2</press>
    <finishing>standard</finishing>
    <urgency>normal</urgency>
  </production>
  - <customer>
    <name>FruitCo</name>
    <number>2712</number>
    <currency>USD</currency>
  </customer>
</job>
```

Document vs. Relational

- Document doesn't require pre-defined schema
- Documents have embedding
- Do support querying for multiple attributes like SQL


```
{
  firstName: "Bob",
  lastName: "Wilson",
  positionTitle: "Manager",
  officeNumber: "2-130",
  officePhone: "555-222-3478",
  hireDate: "1-Feb-2010",
  terminationDate: "12-Aug-2014"
  PreviousPositions: [
    {
      \position: "Analyst",
      StartDate: "1-Feb-2010",
      endDate: "10-Mar-2011"
    } {
      position: "Sr. Analyst",
      startDate: "10-Mar-2011"
      endDate: "29-May-2013"
    } ]
}
```

The database for modern applications

MongoDB is a general purpose, document-based, distributed database built for modern application developers and for the cloud era. No database is more productive to use.

 Try MongoDB free in the cloud!

Get Started

Column Family Databases

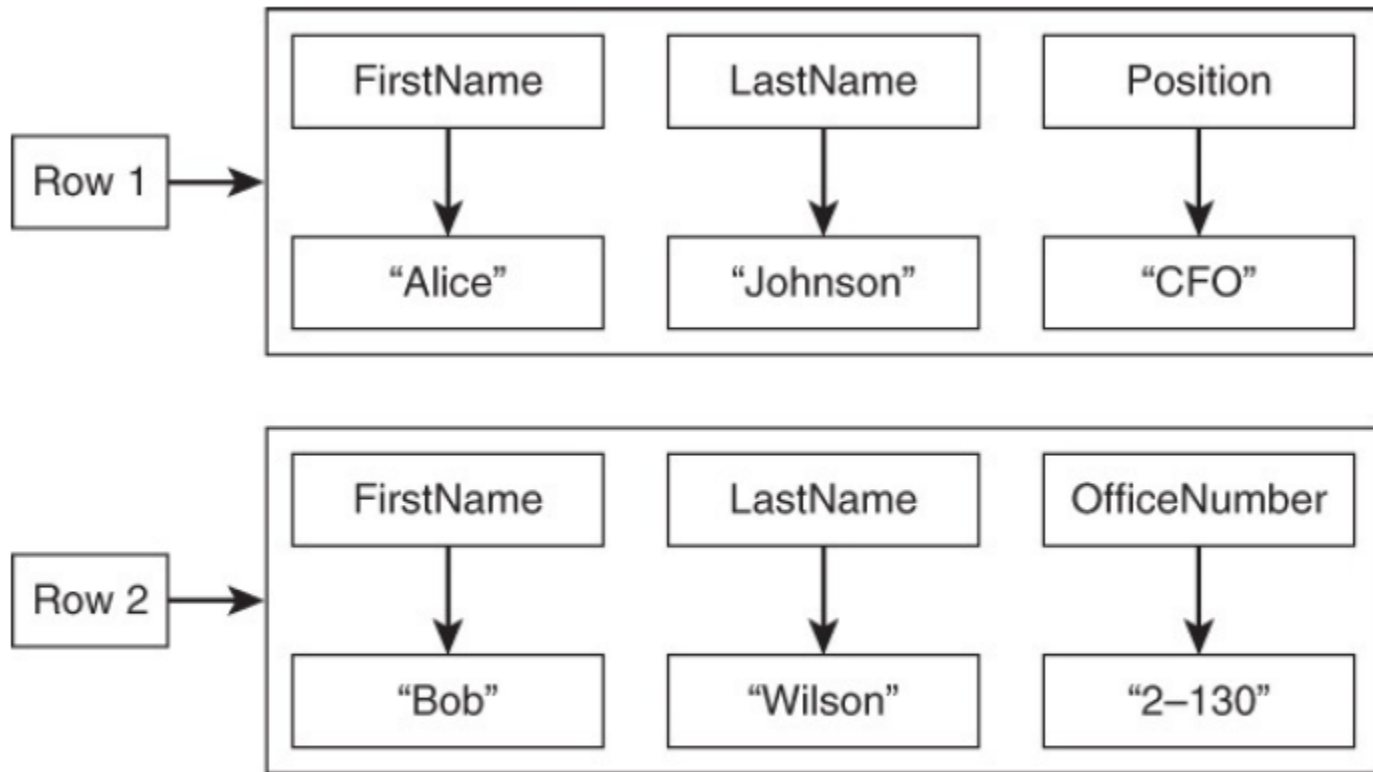


Figure 2.15 *A row consists of one or more columns. Different rows can have different columns.*

Column Family Databases

- Superficially similar to RDBMS
- Query languages look similar except there are no joins
- Unlike RDBMS, sets of columns in a family can vary
- Column DBs are denormalized or structures so that all information is available in one single row
- Primary benefit is that some queries can become extremely fast

Database Products

Contact sales

Get started for free

FORRESTER

Forrester names Google a leader in The Forrester Wave™: Database-as-a-Service, 2019. Get your complimentary report here.

CLOUD BIGTABLE

A petabyte-scale, fully managed NoSQL database service for large analytical and operational workloads.



TRY IT FREE

Low latency, massively scalable NoSQL

Consistent sub 10ms latency

Graph Databases

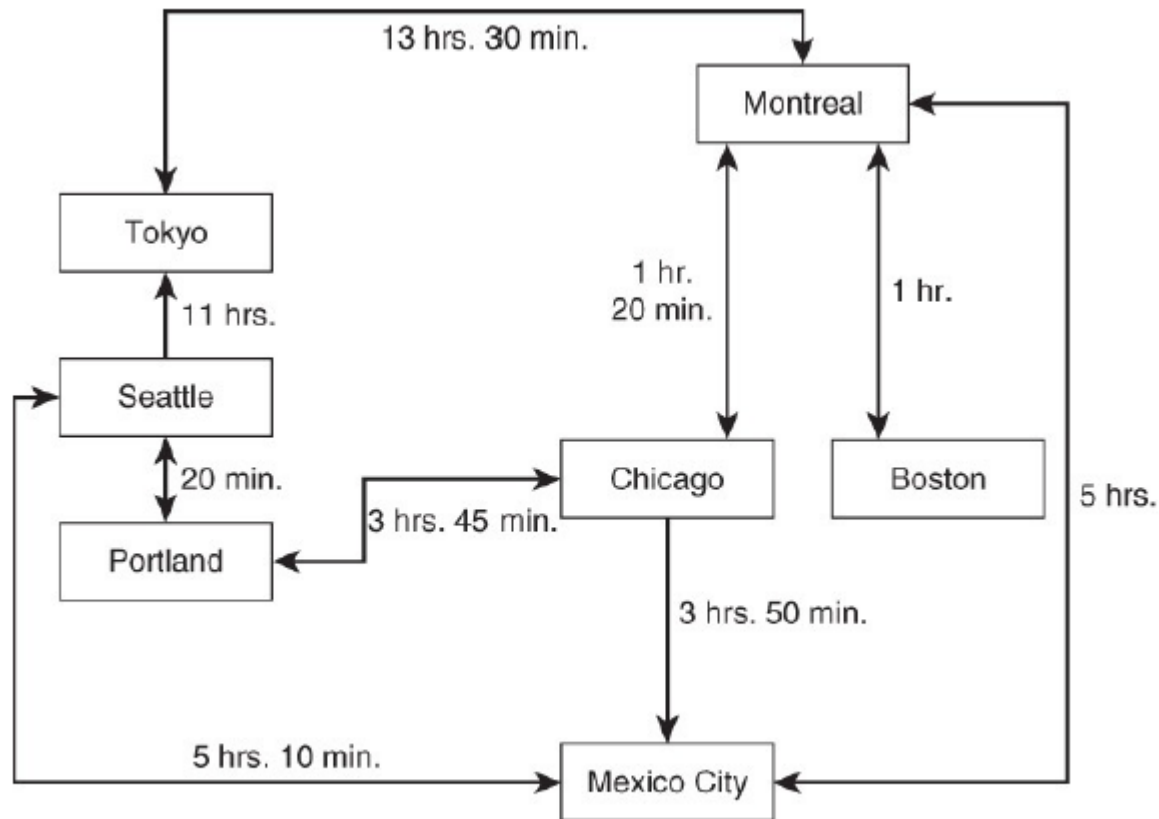


Figure 2.16 Properties of relationships or nodes store attributes about relations between linked nodes. In this case, attributes include flying times between cities.

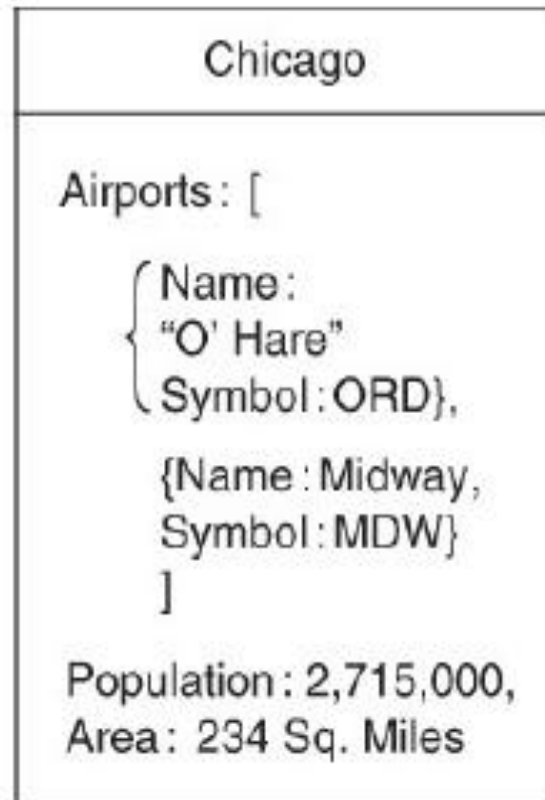


Figure 2.17 Nodes can also have attributes to describe the node. In this case, attributes include information about the airports in the city along with population and geographic area.

Graph Databases vs. Relational

- Graph databases are designed to model adjacency between objects
- Every node in the database contains pointers to adjacent objects in the database
- This allows for fast operations that require following paths through a graph

Find fastest route from Montreal to Mexico City

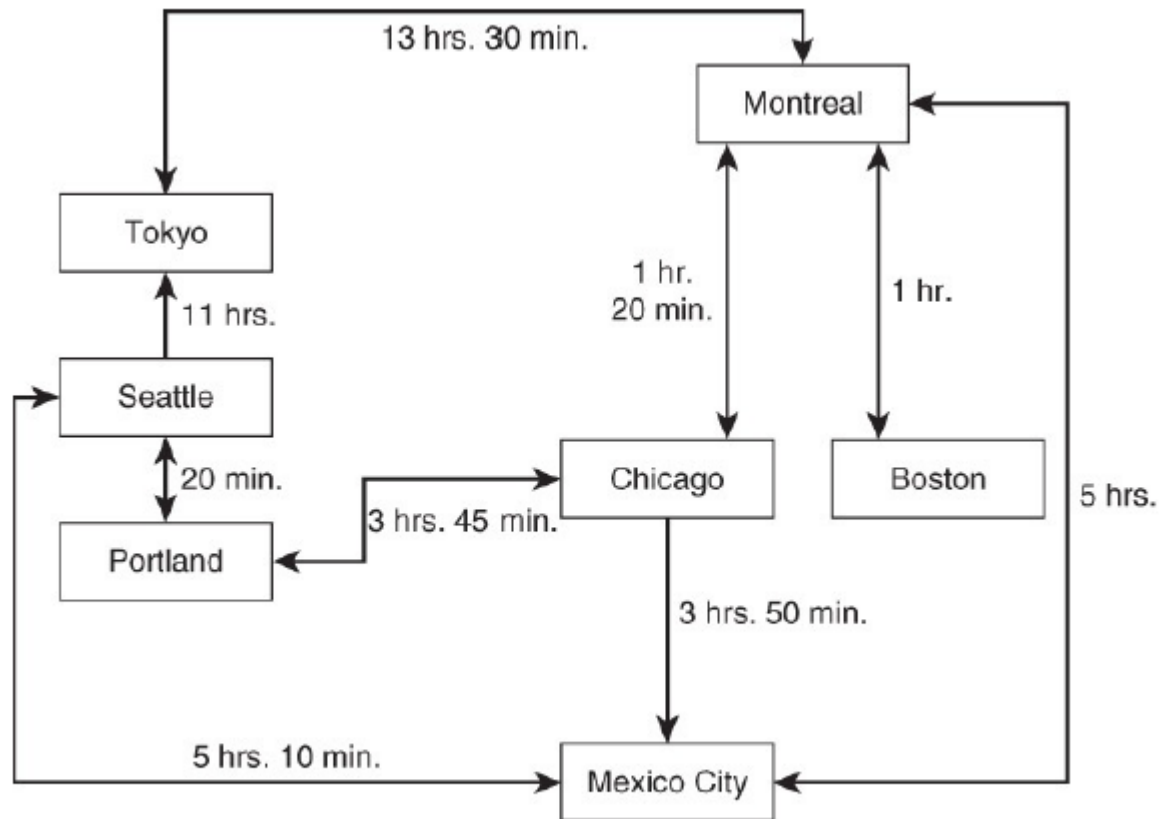


Figure 2.16 Properties of relationships or nodes store attributes about relations between linked nodes. In this case, attributes include flying times between cities.

Now do the same with this RDBMS representation

City 1	City 2	Flying Time
Montreal	Boston	1 hr.
Montreal	Chicago	1 hr. 20 min.
Montreal	Tokyo	13 hr. 30 min.
Montreal	Mexico City	5 hr.
Chicago	Mexico City	3 hr. 50 min.
Chicago	Portland	3 hr. 45 min.
Portland	Seattle	20 min.
Seattle	Tokyo	11 hr.
Seattle	Mexico City	5 hr. 10 min.

Table 2.1 *Flight Times Between Cities Modeled as a Relational Table*

Latest News →

"Something much broader than compensation drives us."—Emil Eifrem, Neo4j Founder—Interview

Company ▾

Support

Contact Us



PRODUCTS

SOLUTIONS

CUSTOMERS

PARTNERS

RESOURCES

DEVELOPERS

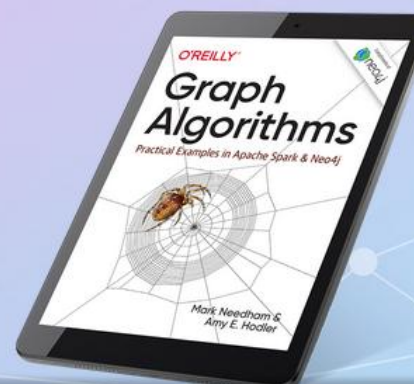
DOWNLOAD NEO4J

Free O'Reilly Book

Graph Analytics For Intelligent Applications

Examples in Spark & Neo4j

Download FREE Book



This website uses cookies

We use cookies to offer you a better browsing experience, analyze site traffic, personalize content and serve targeted ads. Learn about how we use cookies and how you can control them in [Cookie Settings](#). By using our site, you consent to our use of cookies.

ACCEPT COOKIES

Use necessary cookies only



Lecture adapted from chapter 3 of NoSQL for Mere Mortals.